

Text Mining the Great Unread

Language Modeling and the NLP Pipeline

Hilke Reckman

Aarhus University & Unsilo

Tuesday July 26 2016

Natural Language Processing

Progress and opportunities (facilitated by ever faster computers)

Useful everyday applications: Google Search, Siri, autocomplete for SMS, Google Translate, ...

This Lecture:

- ▶ Overview of some of the things you can do with Natural Language Processing
- ▶ Basics of Ngram models
- ▶ Basics of Vector space models

From signal to meaning (and back)

- ▶ Segmentation - split a continuous stream into units
- ▶ Normalization - when things look different, but you want to treat them the same (e.g. spelling variations)
- ▶ Disambiguation - when things look the same, but you want to treat them as different (e.g. *bank*, word senses, N, V)
- ▶ Structural analysis - how do the units combine together?
- ▶ Interpretation

What is a text?

- ▶ A coherent set of signs that transmits some kind of informative message
- ▶ A written representation of a message in natural language
- ▶ Words organized into meaningful sentences, organized into meaningful paragraphs. . .
- ▶ To the computer: **a sequence of bytes that encode characters**
lower case letters, upper case letters, punctuation, spaces, . . .
“Text” $\Rightarrow$ \u0054\u0065\u0078\u0074

The NLP Pipeline: From characters to meaning

- ▶ Tokenization: from characters to words and sentences
- ▶ Normalization: reducing the number of types
- ▶ Part-of-speech tagging
- ▶ (Partial) syntactic processing
- ▶ (Partial) semantic processing

From characters to words

“This is a sentence.”

\u0054\u0068\u0069\u0073\u0020\u0069\u0073\u0020\u0061\u0020\u0073\u0065\u006e\u0074\u0065\u006e\u0063\u0065\u002e

- ▶ Tokenization
 - ▶ Word tokenization: split into words needed for most types of processing
 - ▶ Sentence tokenization: split into sentences
- ▶ Split on white space, punctuation?
- ▶ Keep or discard punctuation?

Tokenization challenges

- ▶ abbreviations:
 - ▶ ‘.’ is part of the word
 - ▶ if the abbreviation is at the end of the sentence, the period does double duty
- ▶ apostrophies, contractions ‘*won’t*’
- ▶ hyphens
- ▶ proper names with unusual characters ‘*M*A*S*H*’
- ▶ inconsistent use of punctuation and white space ‘*lower case*’, ‘*lowercase*’, ‘*lower-case*’
- ▶ compounds (German, Danish, Dutch, ...)
- ▶ multi-word units (words with spaces)
- ▶ emoticons
- ▶ hashtags
- ▶ urls, dates, words containing numbers, ...

[http://nlp.stanford.edu/IR-book/html/htmledition/
tokenization-1.html](http://nlp.stanford.edu/IR-book/html/htmledition/tokenization-1.html)

Tokens, Types, and Normalization

- ▶ Tokens can be sorted into types: distinct forms
- ▶ The token is then an instance of the type
- ▶ Tokens that are identical belong to the same type (but word-sense ambiguity!)
- ▶ Sometimes it makes sense to map tokens that are not identical to the same type
 - ▶ normalize capitalization (common: lowercasing) '*The*', '*the*' but: '*PET*', '*pet*', ambiguity: $A \rightarrow a / \grave{a}$ (fr)
 - ▶ Spelling correction/normalization
 - ▶ Lemmatization: map inflected words to their lemmas (usually dictionary-based) '*use*', '*uses*', '*used*', '*using*'
 - ▶ Stemming: map inflected and derived words to their stems (usually rule-based) '*user*', '*usage*', '*usability*', ...
more reduction than lemmatization, but also more errors
 - ▶ Mapping nominalizations to verbs (Nomlex)
 - ▶ Mapping synonyms (very tricky, but can work in limited cases)

Part of Speech tagging

- ▶ Parts of Speech (PoS): Noun, Verb, Preposition, etc.
- ▶ Often useful in refining search
- ▶ PoS-tagging is a labeling task
- ▶ Disambiguation: identical looking words can have different PoS
- ▶ Use context, e.g previous word, previous PoS-tag
- ▶ Reasonable PoS-tagging systems are available for many languages

PoS-tagging with R and OpenNLP: <http://martinschweinberger.de/docs/articles/PosTagR.pdf>

Multi Word Units, Collocations

- ▶ Words with spaces ('*by and large*', '*San Francisco*')
- ▶ Constructions with syntactic variation ('*kick the bucket*', '*make one's way*', '*call off*')
- ▶ Terms that are more loosely connected

Which terms occur together significantly more often than you would expect if they were distributed randomly over the text?

Syntactic processing: Chunking (Shallow Parsing)

- ▶ Most commonly extracting noun phrases
- ▶ Based on PoS-tag patterns
- ▶ Basis for Named Entity Recognition
- ▶ **I**nside, **O**utside, **B**egin notation

We PRP B-NP
saw VBD O
the DT B-NP
little JJ I-NP
yellow JJ I-NP
dog NN I-NP

Syntactic processing: Full Parsing

- ▶ constituents / dependencies
https://en.wikipedia.org/wiki/Dependency_grammar
- ▶ tree banks (mostly news)
- ▶ grammar engineering
- ▶ resolving attachment ambiguities
- ▶ English: Stanford parser
<http://nlp.stanford.edu:8080/parser/>
- ▶ Works best on heavily edited text, similar to training corpus
- ▶ Expect errors:
 - ▶ rare constructions (especially when syntactically complex)
 - ▶ difficult attachment ambiguity problems (e.g. pp-attachment)
 - ▶ coordination, ellipsis, gapping

One morning I shot an elephant in my pajamas.

How he got into my pajamas I'll never know.

-Groucho Marx

Full Parsing for Text Mining

Only if you have a suitable corpus and a good reason.

- ▶ Very few languages have decent parsers: Modern English, German, Mandarin?, . . .
- ▶ Edited, written language
- ▶ Research questions needs search or analysis of syntactic constructions (e.g. use of relative clauses, semantic roles)
- ▶ Consider work-arounds with PoS-tagging and chunking

Semantic processing

- ▶ Semantic role labeling
- ▶ Named entity recognition
- ▶ Coreference resolution
- ▶ Sentiment analysis
- ▶ Document similarity
- ▶ Word similarity
- ▶ Topic modeling
- ▶ Textual entailment, question answering, information extraction
- ▶ Lexical semantic resources: ontologies, thesauri (e.g. WordNet)

Named Entity Recognition & Coreference Resolution

- ▶ Identify names of people, places, organizations, . . .
- ▶ Identify dates, monetary amounts, . . .
- ▶ Which names refer to the same entities?
- ▶ What do pronouns like '*he*', '*she*', '*it*', '*this*' refer to?

Very useful, but very challenging!

Semantic Role Labeling

- ▶ Identify roles independent of syntactic construction (e.g. object becomes subject in passive)
- ▶ Different types of verbs assign different roles: e.g. agent vs experiencer
- ▶ Needs parsing (or at least advanced chunking)
- ▶ Possible TM relevance: Which entities are depicted as active, which as passive? (political discourse, gender studies, ...)

Sentiment Analysis

- ▶ Detect sentiment expressed in a text: in general or about something specific (a brand, a product, a politician, ...)
- ▶ Often based on lists of positive and negative words
- ▶ Better: phrases (e.g. *'not so nice'* *'Yeah, right!'*)
- ▶ Or machine learning
- ▶ Problems:
 - ▶ Syntactic analysis may be needed: in some languages (e.g. German) negations can be quite far away from the term they are negating
 - ▶ What is sentiment? Some words have positive or negative associations, but are not necessarily always expressions of sentiment (*'death'*)
 - ▶ Ambiguity, e.g. slang expressions like *'sick'*
 - ▶ Irony and sarcasm
 - ▶ Book / movie reviews: distinguish between description of content and evaluation
 - ▶ In machine learning, best predictors sometimes not sentiment words, e.g. certain actor names in movie reviews

Language Representations for Machine Learning

- ▶ Sequential models: ngram models, language as a sequence, probabilistic, predict the next item based on the previous ones
- ▶ Distributional models: count (global or local) occurrences, may completely ignore word order, similarities based on overlap, modeled as a vector space
- ▶ Mixed bag of features: anything that can be extracted and that may be relevant to the problem (typically supervised classification tasks)

N-gram Language Models

unigrams(=current token only, no context), bigrams, trigrams, 4-grams, 5-grams...

- ▶ use the immediately preceding context to predict the next word / character / tag / ...
- ▶ train on a corpus: counts $\rightarrow$ probabilities
- ▶ smoothing: adjust the probabilities in a systematic way, so that things that were absent in the training data are not considered to be impossible
- ▶ for larger values of n , data sparseness becomes an ever bigger problem $\rightarrow$ back-off to simpler models

Generating random text

- ▶ train a model on a genre
- ▶ generate a new text in the same genre by randomly choosing the next word according to the probability

Generation with a word-level model

trained on Sense and Sensibility by Jane Austen

Python NLTK demo

The family of his regard for Marianne listened with increasing surprise . She took them all . They wanted him to do it . " Here comes Marianne ," replied Lucy quickly –" but why should you think he came to town !" cried Marianne again .—" So calm !— so cheerful – when I was simple enough to be overcome . Poor Brandon ! he could spare so considerable a sum with little difficulty , over the contents of the event of that poor girl , without having

Generation with a word-level model

trained on the book of Genesis

Python NLTK demo

In the mount Gilead . And Israel bowed himself upon the wood for the camels , and she , even she herself said , Let me now leave with thee , with bread for all his father would die . And she arose , then shall the LORD . Gad , and the Rephaims , And the servant brought forth by handfuls . And there stood no man with him . And Esau was a cunning hunter , a little way to come in unto her , he ran to meet her , Wilt thou give me a pledge ,

Generation with a character-level model

macbeth, sense and sensibility, the bible

1. The my sigh hose offe. Grane: Whan ces goatherlegaind I bes;
hauethou showad shat be th gon, Off; a
2. Sent king thilever to con aged; af fel could noresion th he hem
taut a sit dow they!" –or broaccelf
3. The at wis knot ye hat tarme: I go ld, to th th poserchiread
wasto wil ight, Thunt unne. 8:6 In Jud

Sparse data and smoothing

- ▶ Sparse: not enough information available about a relevant phenomenon (possibly completely absent)
- ▶ In a text or corpus of any size, most of the words occur only once (Zipf's law)
- ▶ There are no partial occurrences of words
- ▶ Words that occur very few times are typically overrepresented
- ▶ Words that do not occur are underrepresented
- ▶ Unseen words and ngrams are a problem for probabilistic models
- ▶ Smoothing: assign a non-zero probability to unseen events
- ▶ This probability mass needs to come from somewhere, since total probabilities must sum to 1
- ▶ There are many different smoothing methods that do this probability redistribution in different ways

Usage of N-gram models

- ▶ Auto-completion (texting on phones)
- ▶ Machine translation (Is the proposed translation a probable sentence in the target language?)
- ▶ Speech recognition (Is what I think I heard a probably sentence?)
- ▶ Basis of sequence labeling (e.g. PoS-tagging)
- ▶ Text classification (Which model most likely generated this text?)

Google Ngrams

<https://books.google.com/ngrams>

- ▶ Frequencies of word sequences per time period from lots of books
- ▶ For example: find anachronisms in tv-series set in the past

[https:](https://www.techdirt.com/articles/20120616/01535419358/google-books-data-mining-reveals-mad-mens-big-historic.shtml)

[//www.techdirt.com/articles/20120616/01535419358/google-books-data-mining-reveals-mad-mens-big-historic.shtml](https://www.techdirt.com/articles/20120616/01535419358/google-books-data-mining-reveals-mad-mens-big-historic.shtml)

http://www.slate.com/articles/podcasts/lexicon_valley/2012/06/lexicon_valley_anachronisms_in_mad_men_downton_abbey_and_edith_wharton_.html

Vector space models of semantics

From Frequency to Meaning, Turney & Pantel 2010 <https://www.jair.org/media/2934/live-2934-4846-jair.pdf>

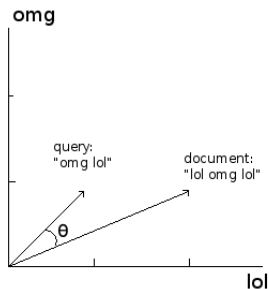
Distributional model:

- ▶ documents that contain similar words have similar meanings
- ▶ words that occur in similar contexts have similar meanings
- ▶ pairs of words that co-occur in similar patterns have similar semantic relations

VSMs for search (Information Retrieval)

- ▶ Latent Semantic Indexing - cluster documents by the words that occur in them
- ▶ document-term matrix
- ▶ bag of words (ignore all structure)

	Ever tried. Ever failed.	No matter. Try again.	Fail again. Fail better.
Ever	2	0	0
tried	1	0	0
failed	1	0	0
No	0	1	0
matter	0	1	0
Try	0	1	0
again	0	1	1
Fail	0	0	2
better	0	0	1



Linguistic processing for VSMs

- ▶ Tokenization (or character ngrams)
- ▶ Ignore 'stop words'
- ▶ Normalization: case folding, stemming
- ▶ Annotation: PoS-tagging, word sense tagging, parsing (depends on application)

Mathematical processing for VSMs

- ▶ Weighted instead of raw frequencies
more weight to terms that distinguish a document from others:
 - ▶ basic:tf/idf
 - ▶ better: PPMI
 - ▶ chi square? (good for collocations)
- ▶ Remove rare terms
- ▶ Smoothing the frequencies
- ▶ Smoothing the matrix (SVD, etc.): Matrix math tricks
- ▶ Measuring distances: Cosine distance (+ techniques to improve efficiency)

Word2vec

- ▶ Mikolov et. al. (Google)
- ▶ word - context matrix
- ▶ word similarities
- ▶ calculate relations with vectors
(king - man + woman = queen)
- ▶ playground demo: <http://deeplearner.fz-qqq.net/>

Applications of VSMs

- ▶ Document retrieval / clustering / classification
- ▶ Document segmentation
- ▶ Question answering
- ▶ Word similarity / clustering / classification
- ▶ Automatic thesaurus generation
- ▶ Word sense disambiguation
- ▶ Spelling correction
- ▶ Textual advertising
- ▶ Information extraction (e.g. named entity recognition)
- ▶ Relational similarity / clustering / classification / search
- ▶ Pattern similarity
- ▶ Grammar induction

Text Mining with a VSM and a seed list

- ▶ Make a list of terms related to a concept/topic that you are interested in
- ▶ Use a VSM to find similar terms and expand the list
- ▶ Use the expanded list to extract passages for further study
- ▶ Refine if needed

Concluding remarks

- ▶ Many modern approaches in NLP use ngram probabilities, vector space models, or a combination of both
- ▶ Segmentation is needed for most task (and will not be perfect)
- ▶ Normalization can help with sparse data, but will also introduce errors
- ▶ Annotation/disambiguation can help make useful distinctions, but is not error-free either
- ▶ Using PoS-tagging is often a good idea, as well as extracting collocated terms
- ▶ More advanced syntactic and semantic processing technologies can be tempting but are often not mature and should be used with care